JAVA OOP LABORATORY EXERCISE

CAMPUS CAFÉ ORDERING SYSTEM

Student Name: Aira Jefferson T. Cabano

Section: Section E

Subject: Object-Oriented Programming

Date: November 10, 2026

Introduction

The Campus Café Ordering System is a Java console application that processes customer orders. Each order can contain up to three menu items: a beverage, pastry, or meal. The program calculates item prices according to their options, applies a 10% discount to student customers, accepts a voucher amount, and awards loyalty points based on the final amount paid.

Program Structure

The project contains seven Java classes. MenuItem is the abstract parent class. Beverage, Pastry, and Meal inherit from it. Customer stores customer information and loyalty points. Order connects a customer with up to three MenuItem objects. CafeApp contains the main method and demonstrates the program.

OOP Concepts Used

Conditional statements. if and if-else statements validate values and make decisions, including negative-price checks, beverage size pricing, warming and upsize fees, the three-item limit, and student discounts.

Classes and objects. Seven classes organize the program. CafeApp creates Customer, Beverage, Pastry, Meal, and Order objects to perform the required tasks.

Object interaction. Order stores a Customer and up to three MenuItem objects. It calls each item's computePrice() method and updates the customer's loyalty points.

Constructors and constructor overloading. MenuItem and its subclasses have default and parameterized constructors. Customer has three constructors: Customer(), Customer(String), and Customer(String, boolean).

Encapsulation and data hiding. Instance variables are private or protected. Getters and setters provide controlled access, and setters validate values such as prices, sizes, extra sides, and loyalty points.

Method overloading. Customer uses addPoints(int) and addPoints(double). Order uses computeTotal() and computeTotal(double) to calculate totals with or without a voucher.

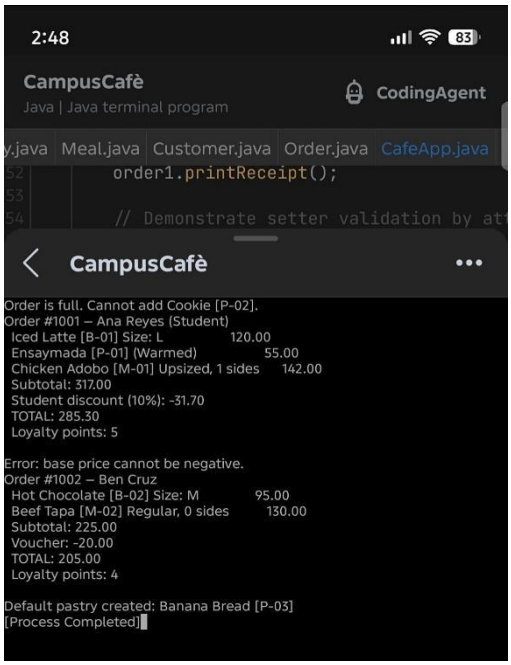
Method overriding and polymorphism. Beverage, Pastry, and Meal override computePrice() and getDescription().

Order stores them as MenuItem references, so the correct subclass method runs at runtime.

Abstraction. MenuItem is abstract and declares computePrice() as an abstract method. Each concrete menu-item subclass implements its own pricing rule.

Sample Program Output

This image was generated from the compiled program's actual console output.



Challenges Encountered and Solutions

Different item prices: The abstract computePrice() method is overridden in each menu-item subclass so each type can apply its own pricing rules.

Invalid values: Conditional statements in setters reject negative prices, negative extra sides, invalid beverage sizes, and negative loyalty point values.

Three-item limit: Order uses three individual MenuItem fields and an item counter. addItem() checks for an available slot before accepting another item.

Discounts, vouchers, and points: Order calculates the subtotal, applies a student discount, subtracts a valid voucher without allowing a negative total, and awards loyalty points once.

Conclusion

The Campus Café Ordering System demonstrates the Java OOP concepts required in the laboratory exercise:

abstraction, inheritance, encapsulation, polymorphism, method overloading, constructors, and conditional statements. It follows the instruction to use individual item fields instead of arrays or Java collection classes.

How to Compile and Run

javac MenuItem.java Beverage.java Pastry.java Meal.java Customer.java Order.java CafeApp.java java CafeApp